NAME: SABITHA C
REG NO: 2303412510422307

**Ex.No.:1 HADOOP INSTALLATION ONE NODE HADOOP CLUSTER**
**Date:**

**AIM:**
To install hadoop environment in ubuntu.

**PROCEDURE:**
1. Unzip the files jdk and hadoop.
2. Set the path of jdk and hadoop in /etc/profile file
3. Check the updation of file by source command.
4. Update 5 files as soon in the below command.
5. Format the path.
6. Start the dfs and check the no. of nodes running.
7. Start the yarn and check the no. of nodes running.
8. Open the browser and check whether the hadoop is installed correctly.
9. Add a file and check whether we can view the file.
10. After completing the process stop dfs and yarn properly.

**COMMANDS:**

1. **cloud@ubuntu:~$ ls**
   Desktop Downloads exp2.odt oddeven Public Videos
   Documents exp1.odt Music
   PicturesTemplatesVirtualBoxVMs
2. **cloud@ubuntu:~$ cd Downloads**
3. **cloud@ubuntu:~/Downloads$tar zxvf jdk-8u60-linux-x64.gz**
4. **cloud@ubuntu:~/Downloads$ cd jdk1.8.0_60/**
5. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$ pwd**
   /home/cloud/Downloads/jdk1.8.0_60 - COPY THE PATH
6. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$sudonano/etc/profile**

A file opens and after the below two lines add in last lines

```
Type the lines as follows
JAVA_HOME=/home/cloud/Downloads/jdk1.8.0_60
PATH=$PATH:$JAVA_HOME/bin
export PATH JAVA_HOME
```

7. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$ source /etc/profile**
8. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$sudo apt-get install openssh-server**

   [sudo] password for cloud:
9. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$ ssh-keygen**

   Press enter key, no need to type enter key.
10. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$ssh-copy-id -i localhost**

cloud@localhost's password:

11. **cloud@ubuntu:~/Downloads/jdk1.8.0_60$ cd ..**
12. **cloud@ubuntu:~/Downloads$tar zxvf hadoop-2.7.0.tar.gz**
13. **cloud@ubuntu:~/Downloads$ cd hadoop-2.7.0/**
14. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ pwd**

/home/cloud/Downloads/hadoop-2.7.0 - COPY THE PATH

15. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$sudo nano /etc/profile**

A file opens and after the below two lines added in last line

```
Type the lines as follows
JAVA_HOME=/home/cloud/Downloads/jdk1.8.0_60
HADOOP_PREFIX=/home/cloud/Downloads/hadoop-2.7.0

PATH=$PATH:$JAVA_HOME/bin
PATH=$PATH:$HADOOP_PREFIX/bin
export PATH JAVA_HOME HADOOP_PREFIX
```

16. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$source /etc/profile**
17. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ cd etc**
18. **cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc$ cd hadoop**
19. **cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc/hadoop$ nano hadoop-env.sh**

```
export JAVA_HOME=/home/cloud/Downloads/jdk1.8.0_60
export HADOOP_PREFIX=/home/cloud/Downloads/hadoop-2.7.0
```

**20. cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc/hadoop$nano core-site.xml**

```
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://localhost:9000</value>
</property>
</configuration>
```

**21 .cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc/hadoop$ nano hdfs-site.xml**

Inside the configuration tag insert the following line

```
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>
```

**22. cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc/hadoop$nano mapred-site.xml**

Inside the configuration tag insert the following lines

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
</configuration>
```

**23. cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc/hadoop$ nano yarn-site.xml**

```
<configuration>
<!-- Site specific YARN configuration properties -->
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>
```

**24. cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc/hadoop$cd..**

**26. cloud@ubuntu:~/Downloads/hadoop-2.7.0/etc$ cd ..**

**27. cloud@ubuntu:~/Downloads/hadoop-2.7.0$bin/hadoop namenode-format**

**28. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ sbin/start-dfs.sh**

**29. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ jps**

5632 DataNode
5428 NameNode
5979 Jps
5851 SecondaryNameNode

**30. cloud@ubuntu:~/Downloads/hadoop-2.7.0$sbin/start-yarn.sh**

**31. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ jps**

5632 DataNode
6209 NodeManager
6050 ResourceManager
5428 NameNode
6522 Jps
5851 SecondaryNameNode

**32.** Open a browser and type as: http:\\localhost:50070

**33. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hdfs dfs -mkdir /user1**

**34. Open a new terminal cloud@ubuntu:~/Downloads/hadoop-2.7.0$ cd .. cloud@ubuntu:~/Downloads$tar zxvf mrsampledata.tar.gz**

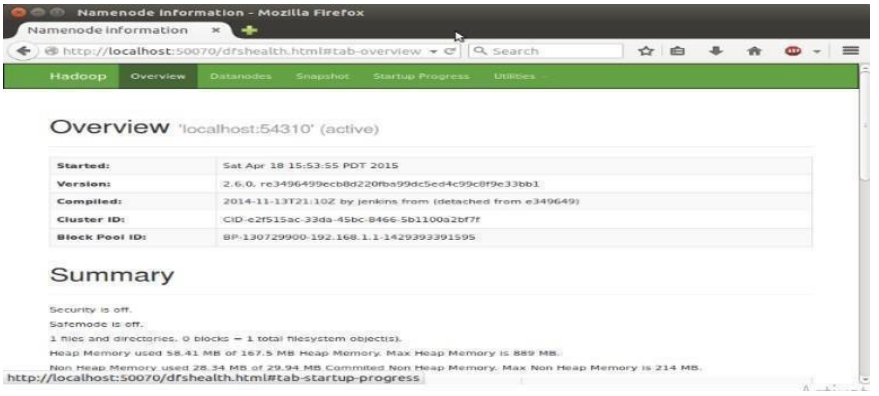
file1.txt
file2.txt
file3.txt
file4.txt
file5.txt

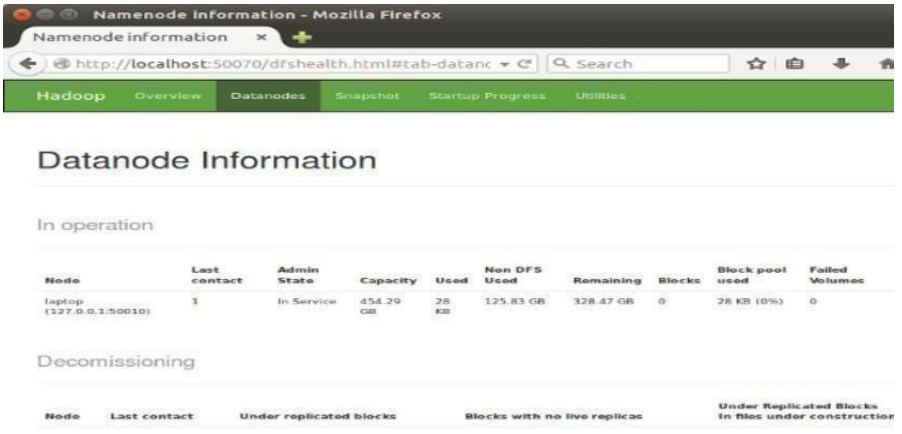
NAME: SABITHA C
REG NO: 2303412510422307

35. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hdfs dfs -put ../file1.txt /user1**
Inside the browser. Click the folder name user1 and you can see the file1.txt file inside the user1 folder

36. **After completing all the process,stop the dfs and yarn as follows:**
**cloud@ubuntu:~/Downloads/hadoop-2.7.0$ sbin/stop- yarn.sh**
**cloud@ubuntu:~/Downloads/hadoop-2.7.0$ sbin/stop- dfs.sh**

**OUTPUT:**




**RESULT:**
Thus hadoop is installed in the physical machine and executed successfully.

**NAME: SABITHA C**
**REG NO: 2303412510422307**

**Ex. No.:2 WORD COUNT PROGRAM - USING MAP AND REDUCE TASK**
**Date:**

**AIM:**
To execute word count program using map and reduce task.

**PROCEDURE:**
1. Format the path.
2. Start the dfs and check the no. of nodes running.
3. Start the yarn and check the no. of nodes running.
4. Openthe browser and check whether the hadoop is installed correctly.
5. Add a file and check whether we can view the file.
6. Implement the grep command for the file added and see the result.
7. Implement the wordcount command for the file added and see the result.
8. After completing the process stop dfs and yarn properly.

**COMMANDS:**

1. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$bin/hadoop namenode-format**
2. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ sbin/start-dfs.sh**
3. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ jps**

5632 DataNode
5428 NameNode
5979 Jps
5851 SecondaryNameNode

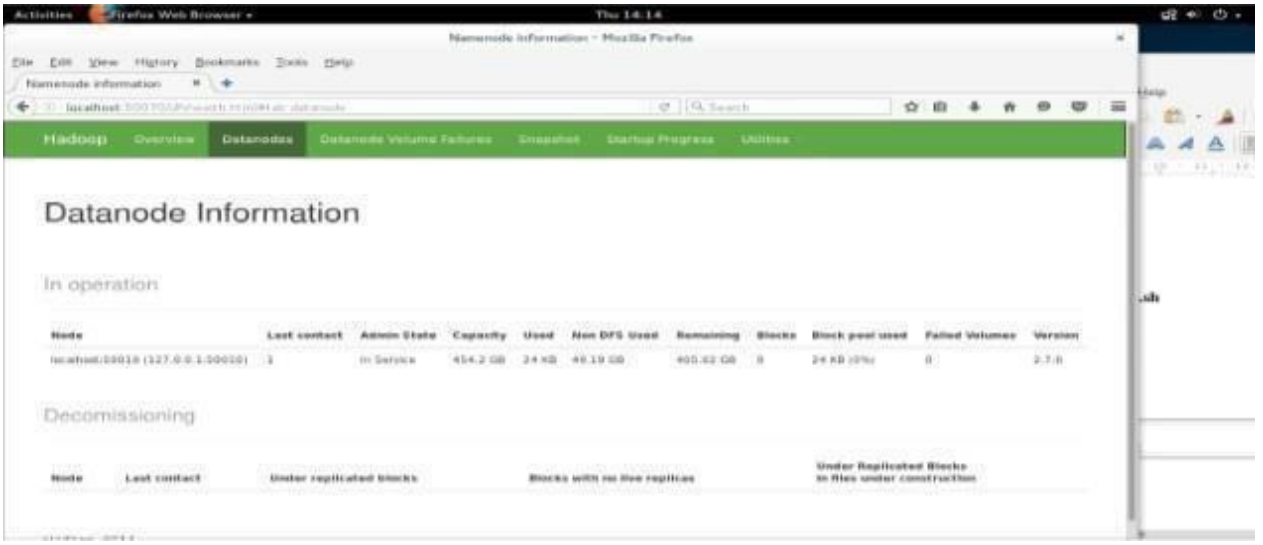
4. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$sbin/start-yarn.sh**

5. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ jps**

5632 DataNode
6209 NodeManager
6050 ResourceManager
5428 NameNode
6522 Jps
5851 SecondaryNameNode

6. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$ sbin/stop-dfs.sh**
7. **cloud@ubuntu:~/Downloads/hadoop-2.7.0$sbin/stop-yarn.sh**

**Open a browser and type as: http:\\localhost:50070**
Choose DataNode. U can see a single node created in it U can see the folder created inside utilities tab.




**8. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hdfs dfs -mkdir /user1**




Open a new terminal

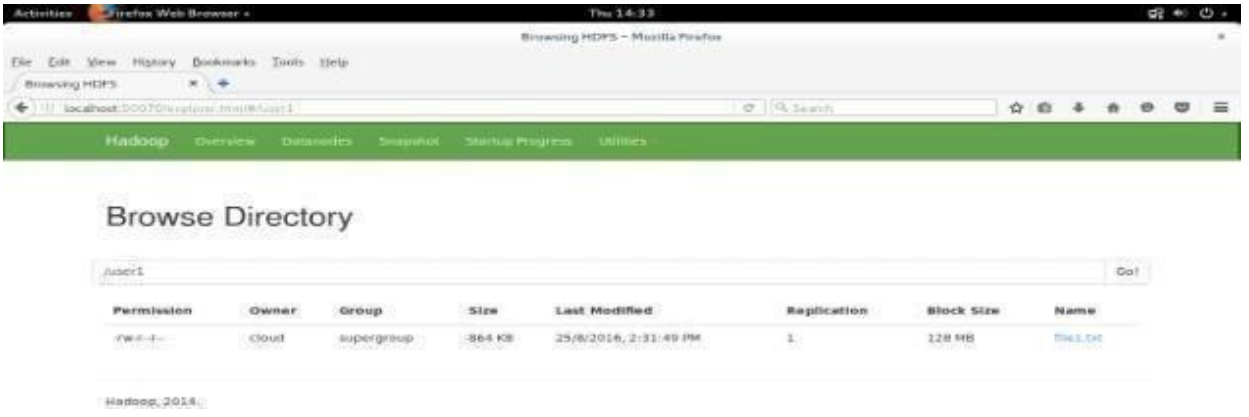
**10. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ cd ..**

**11. cloud@ubuntu:~/Downloads$tar zxvf mrsampledata.tar.gz**

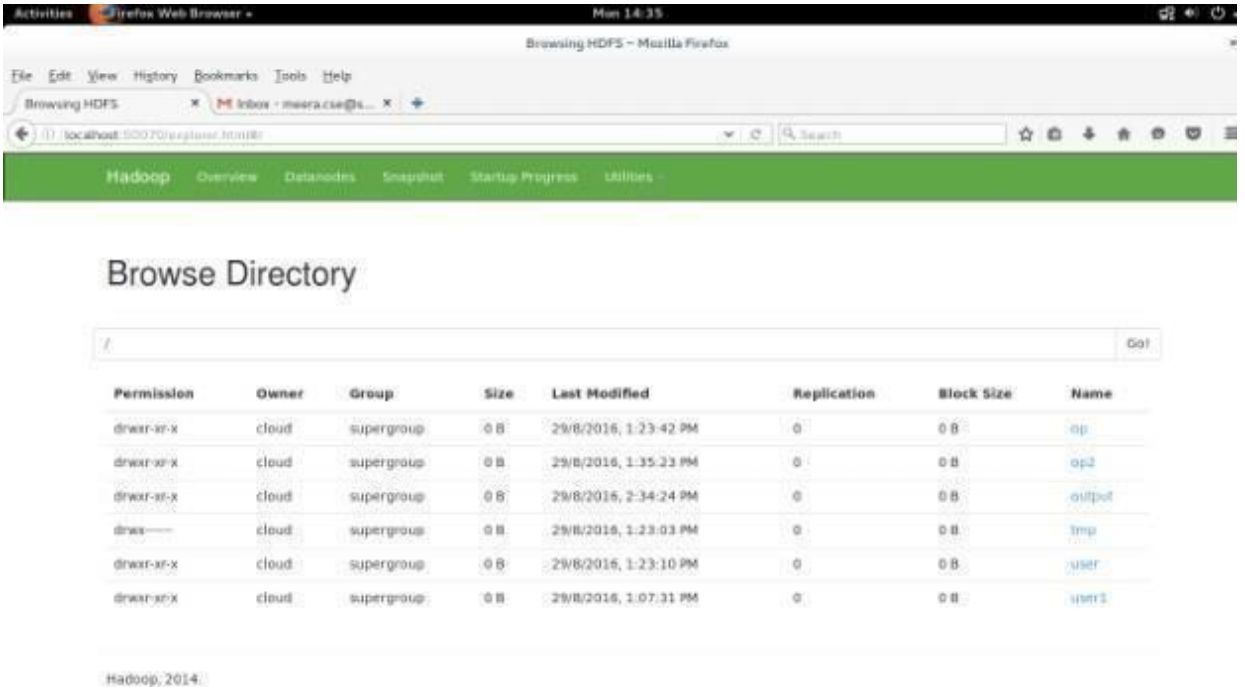
file1.txt
file2.txt
file3.txt
file4.txt
file5.txt

**12. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hdfs dfs -put ../file1.txt /user1**

**13.** Inside the browser.Click the folder name user1 and you can see the file1.txt file inside the user1 folder




**14. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-examples-2.7.0.jargrep/user 1/*/output '(CSE)'**




**15. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-examples-2.7.0.jar wordcount/user1/file1.txt**
**/output1**

**16. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hdfs dfs -cat /output/***
9894 CSE

**17. cloud@ubuntu:~/Downloads/hadoop-2.7.0$ bin/hdfs dfs -cat /output1/***

B.ARCH 9864
B.TECH(BIO) 9964
B.TECH(IT) 10000
BE(AME) 9853
BE(CIVIL) 10043
BE(CSE) 9894
BE(ECE) 10048
BE(EEE) 9937
BE(ICE) 9872
BE(MECH) 9873

Youcanalsoseetheouputby

1) output

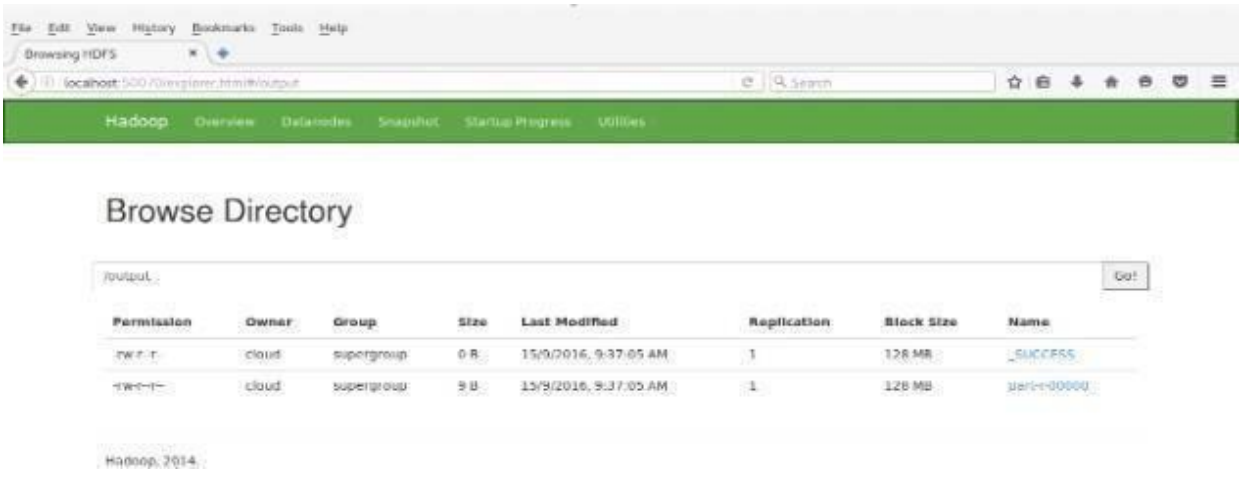
Inside Browser Directory

**output-->part-r-00000-->ClickDownload**

2) output1

Inside Browser Directory

**output-->part-r-00000-->ClickDownload**




**RESULT:**

Thus the wordcount program using map and reduce task is executed successfully.

NAME: SABITHA C
REG NO: 2303412510422307

## EX. NO: 3 IMPLEMENT WORD COUNT/ FREQUENCY PROGRAMS USING MAPREDUCE

**DATE:**

**AIM:**

To write a java program for counting the number of occurrences of each word in a text file using the Mapreduce concepts.

**ALGORITHM:**

1. Open an editor andt ype WordCountprogram and save as WordCount.java
2. Set the path as export HADOOP_CLASSPATH=${JAVA_HOME}/lib/tools.jar
3. To compile the program,bin/hadoopcom.sun.tools.javac.MainWordCount.java Create a jar file, jar cf wc.jar WordCount*.class
4. Create input files input.txt,input1.txt and input2.txt and create a directory in hdfs, /mit/wordcount/input
5. Move these i/p files to the hdfs system, bin/hadoopfs –put /opt/hadoop-2.7.0/input.txt /mit/wordcount/input/input.txt repeat this step for other two i/p files.
6. To execute,bin/hadoopjarwc.jarWordCount/mit/wordcount/input/mit/wordcount/output .
7. The mapreduce result will be available in the output directory.

**PROGRAM CODING:**

hduser@sairam-VirtualBox:/usr/local/hadoop/bin$nanowordcount7.java import

java.io.IOException;

import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration; import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job; import
org.apache.hadoop.mapreduce.Mapper; import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat; import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

```
public class wordcount7 {

public static classTokenizerMapper extends Mapper<Object,Text,Text,IntWritable>

private final static Int Writableone=new IntWritable(1);

private Textword=new Text();

public void map(Objectkey,Textvalue,Contextcontext)throws IOException,
                    InterruptedException

{String Tokenizeritr=new StringTokenizer(value.toString());


while (itr.hasMoreTokens())

 {

word.set(itr.nextToken());context.write(word,one);}}}


public static class Int SumReducer extends Reducer<Text,IntWritable,Text,IntWritable>\

 {

private Int Writableresult=new IntWritable();

public void reduce(Textkey,Iterable<IntWritable>values,Contextcontext)throws
                    IOException, InterruptedException

{

int sum = 0;

for(Int Writableval:values){ sum += val.get(); }

result.set(sum);context.write(key,result);}

}

public static void main(String[] args)throws Exception

 { Configuration conf = new Configuration(); Job job=Job.getInstance(conf,"wordcount");
job.setJarByClass(wordcount7.class); job.setMapperClass(TokenizerMapper.class);
job.setCombinerClass(IntSumReducer.class); job.setReducerClass(IntSumReducer.class);
job.setOutputKeyClass(Text.class); job.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
```

```
FileOutputFormat.setOutputPath(job,newPath(args[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
} }
```

**INPUT FILE:**

wc1.txt

**OUTPUT:**

```
/mit/wordcount/input 2
/mit/wordcount/input/input.txt  1
/mit/wordcount/output. 1
/opt/hadoop-2.7.0/input.txt 1
1. 1
2. 1
3. 1
4. 1
5. 1
6. 1
7. 1
8. 1
Create 2
HADOOP_CLASSPATH=${JAVA_HOME}/lib/too
ls.jar1 Move 1
Open 1
STEPS: 1
Set 1
T
h
e
1
T
o
2
WordCount 2
WordCount*.class1
WordCount.java2
a 2
an 1
and 4
as 2
```

available 1
be 1
bin/hadoop 3
cf 1
com.sun.tools.javac.Main 1
compile 1
create 1
direct
ory 1
direct
ory.1
editor
1
execute, 1
export 1
file, 1
files 2
files. 1
for 1
fs 1
hdfs 1
hdfs, 1
i/p 2
in 2
input 1
input.txt,input1.txt 1
input2.txt 1
jar 3
mapreduce 1
other 1
output 1
path 1
program 1
program, 1
repeat 1
result 1
save 1
step 1
system, 1
the 3
these 1

this 1
to 1
two 1
type 1

**RESULT:**

Thus the java program for word count using mapreduce has been implemented successfully.

**EX.NO: 4 IMPLEMENT AN MR PROGRAM THAT PROCESSES A WEATHER DATASET**

**DATE**:

**AIM:**

To write an MR program that processes a weather dataset.

**PROGRAM CODING:**

```
import
java.io.BufferedReader;
import java.io.FileReader;
import
java.io.IOException;

public classWeatherAnalyzer{
   public staticvoidmain(String[]
      args){ if (args.length != 1) {
         System.err.println("Usage:WeatherAnalyzer<inputfile>");
         System.exit(1);
      }
      String inputFile = args[0];
      try(BufferedReaderreader=newBufferedReader(newFileReader(inputFile))){
         String line;
         int
         maxTemperature=Integer.MIN_VALU
         E; int
         minTemperature=Integer.MAX_VALU
         E;

         while((line=reader.readLine())!=null){
            String[] tokens = line.split("\t");
            if (tokens.length >= 2) {
               int temperature = Integer.parseInt(tokens[1]);

               if(temperature>maxTempera
                  ture){ maxTemperature =
                  temperature;
               }
```

```
            if(temperature<minTemperature){  minTemperature
               = temperature;
            }
          }
        }
        System.out.println("MaximumTemperature:"+maxTemperat
        ure); System.out.println("Minimum Temperature: " +
        minTemperature);

      } catch (IOException e) {
        System.err.println("Errorreadinginputfile:"+
        e.getMessage());
      } catch (NumberFormatException e) {
        System.err.println("Invalidtemperaturevalue:"+
        e.getMessage());
      }
    }
}
```

**OUTPUT:**

whether output directory is in place on HDFS. Execute following command to verify the same.

**hduser1@ubuntu:/usr/local/hadoop2.6.1/bin$** ./hadoop fs -ls
/user/hduser1/testfs/output_mapre
d3 Found 8 items
-rw-r--r--3zytham supergroup 4382015-12-1119:21
/user/hduser1/testfs/output_mapred3/**Austin-r-00000**
-rw-r--r--3zytham supergroup 2192015-12-1119:21
/user/hduser1/testfs/output_mapred3/**Baltimore-r-00000**
-rw-r--r--3zytham supergroup 2192015-12-1119:21
/user/hduser1/testfs/output_mapred3/**Boston-r-00000**
-rw-r--r--3zytham supergroup 5112015-12-1119:21
/user/hduser1/testfs/output_mapred3/**California-r-00000**
-rw-r--r--3zytham supergroup 1462015-12-1119:21
/user/hduser1/testfs/output_mapred3/**Newjersy-r-00000**
-rw-r--r--3zytham supergroup 2192015-12-1119:21
/user/hduser1/testfs/output_mapred3/**Newyork-r-00000**
-rw-r--r--3zytham supergroup 0 2015-12-11 19:21
/user/hduser1/testfs/output_mapred3/_SUCCESS

-rw-r--r--3zytham supergroup 0 2015-12-11 19:21
/user/hduser1/testfs/output_mapred3/part-r-00000
**hduser1@ubuntu:/usr/local/hadoop2.6.1/bin$** ./hadoop fs -cat
/user/hduser1/testfs/output_mapred3/Austin-r-00000
25-Jan-2018Time:12:34:542MinTemp:-22.3Time:05:12:345MaxTemp:35.7
26-Jan-2018Time:22:00:093MinTemp:-27.0Time:05:12:345MaxTemp:55.7
27-Jan-2018Time:02:34:542MinTemp:-22.3Time:05:12:345MaxTemp:55.7

**RESULT:**

Thus the implementation of an MR program that processes a weather dataset has been implemented successfully.

**EX.NO: 5A IMPLEMENT AN APPLICATION USING HIVE**

**DATE:**

**AIM:**

To implement an application using Hive

**ALGORITHM:**

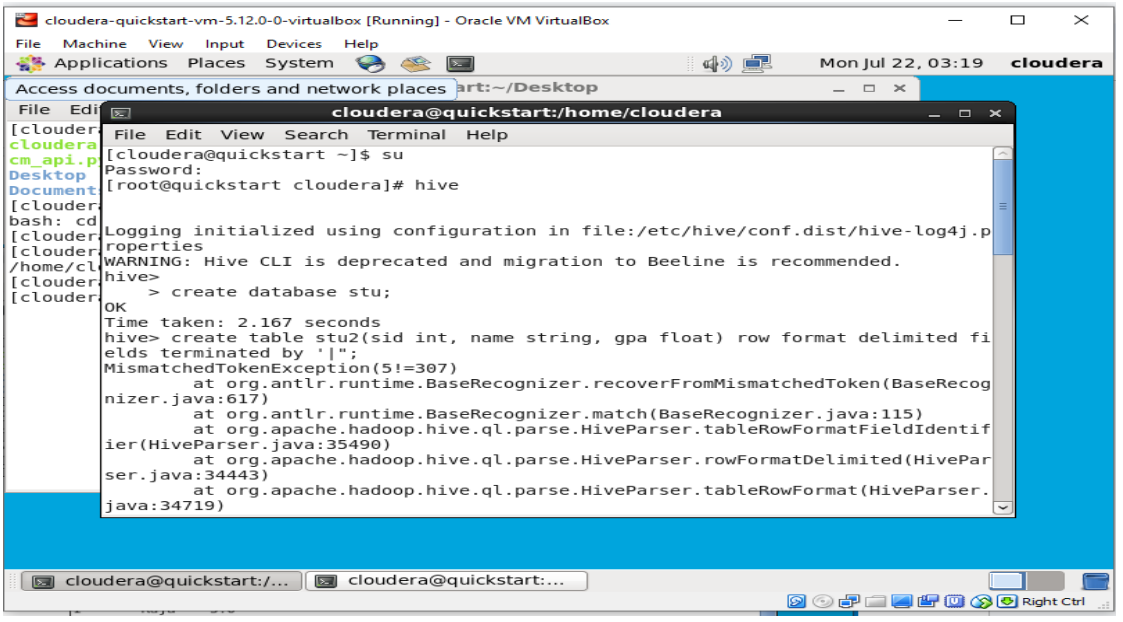
Step 1: Start the Cloudera Virtual Machine.
Step 2: Open the terminal and start Hive.
Step 3: Create a database named stu.
Step 4: Create a table stu2 with the fields sid, name, and gpa.
Step 5: Store the student details in a file named stu1.
Step 6: Load the data from the local file stu1 into the stu2 table.
Step 7: Display the available databases using SHOW DATABASES.
Step 8: Display the records in the stu2 table using SELECT *.
Step 9: Verify the student details are displayed correctly.
Step 10: Stop the execution.

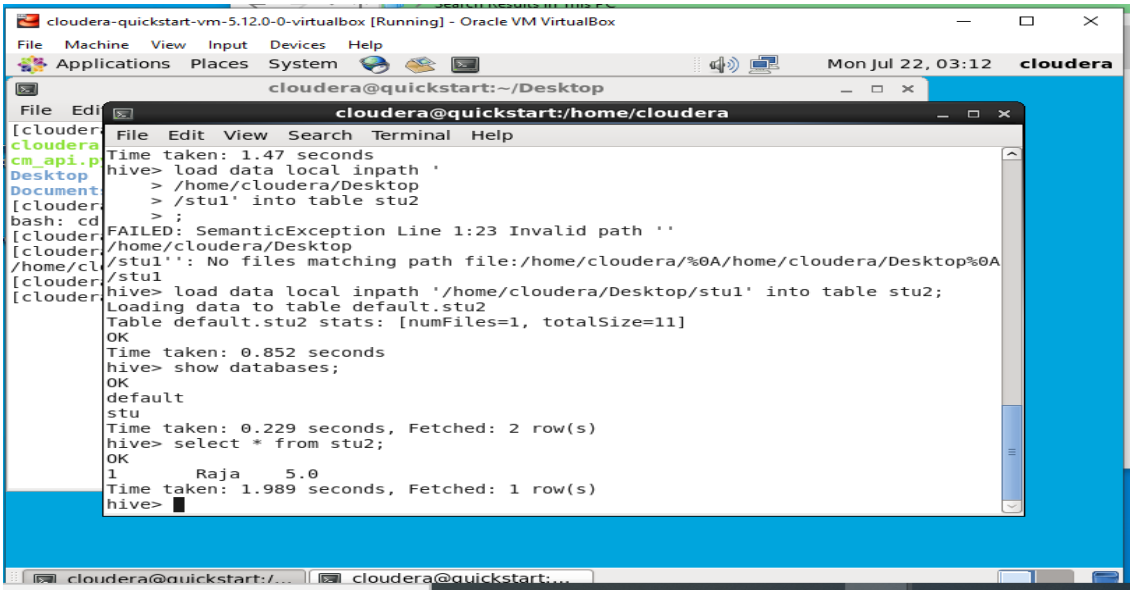
**PROGRAM:**

```
hive> create database stu;
OK
Time taken:.....
hive> create table stu2(sid int, name string, gpa float) row format delimited fields
terminated by '|';
OK
Time taken.....
hive> load data local inpath '/home /coudera/Desktop/stu1' into table stu2;
Ok
hive> show databases;
OK
default
stu
hive> select * from stu2;
OK
1  Raja  5.0
```

NAME: SABITHA C
REG NO: 2303412510422307

## OUTPUT:




## RESULT:

Thus the implementation of the hive application is done.

NAME: SABITHA C
REG NO: 2303412510422307

## EX.NO: 5B IMPLEMENT AN APPLICATION USING HIVE/PIG

**DATE:**

**AIM:**

To implement word count using PIG

**ALGORITHM:**

Pig Latin provides four different types of diagnostic operators−

- Dump operator
- Describe operator
- Explanation operator
- Illustration operator

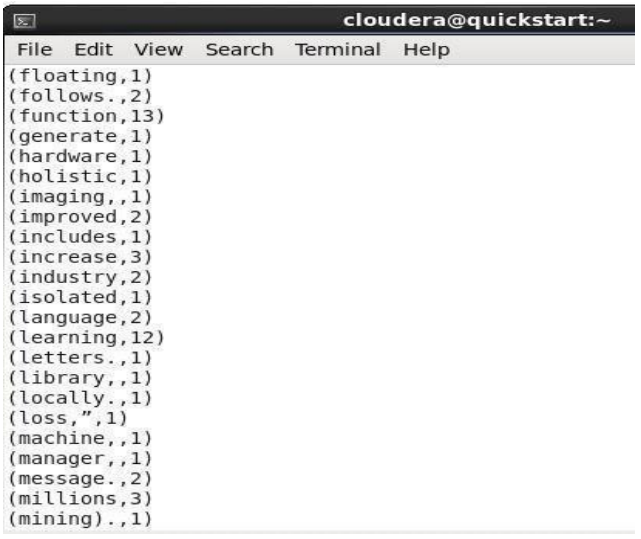
**PROGRAM:**

Word Count Example Using PigScript:

```
lines = LOAD '/user/hadoop/HDFS_File.txt' AS (line:chararray);
words=FOREACH lines GENERATE
FLATTEN(TOKENIZE(line))asword; grouped = GROUP words
BY word;
wordcount=FOREACH grouped GENERATE
group,COUNT(words); DUMPwordcount;
```

The above pig script first splits each line into words using the TOKENIZE operator. The tokenize function creates a bag of words. Using the FLATTEN function, the bag is converted into a tuple. In the third statement, the words are grouped together so that the count can be computed which is done in the fourth statement.

**OUTPUT:**




**RESULT:**

The pig script was successfully executed.

NAME: SABITHA C
REG NO: 2303412510422307

## EX. NO:6 VISUALIZE DATA USING ANY PLOTTING

**FRAMEWORK DATE:**

**AIM:**

To write a code for visualizing data using any plotting framework

**IMPLEMENTATION:**

**ggplot2:**With these R basics in place,let's dive into the ggplot2 package.

**Installation:**

One of R's greatest strengths is its excellent set of packages. To install a package, you can use the install.packages() function**.install.packages("ggplot2")**

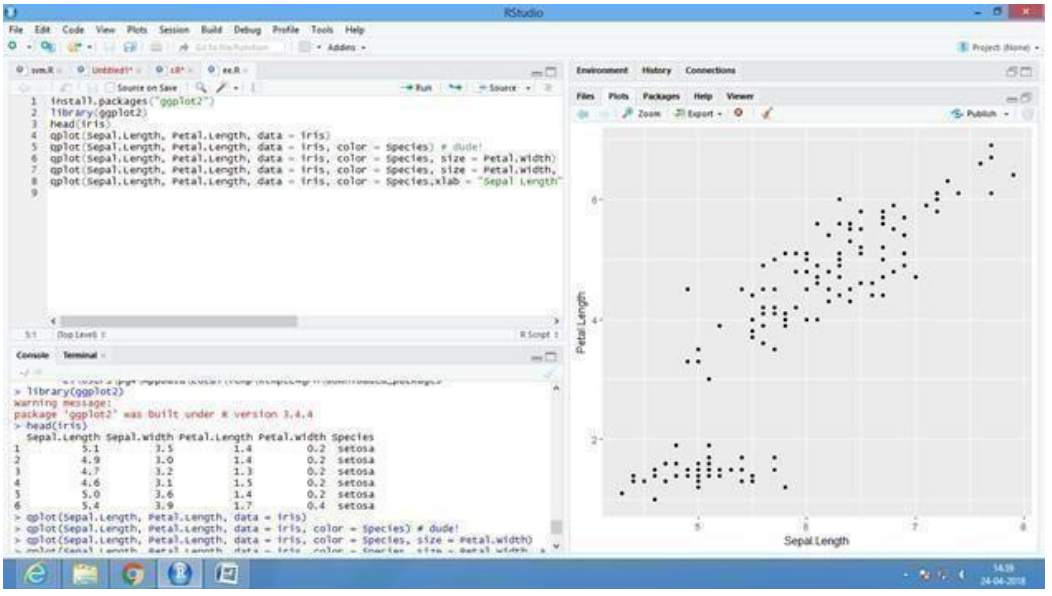
To load a package into your current R session,uselibrary().**library(ggplot2)**

**Scatterplots with qplot()**

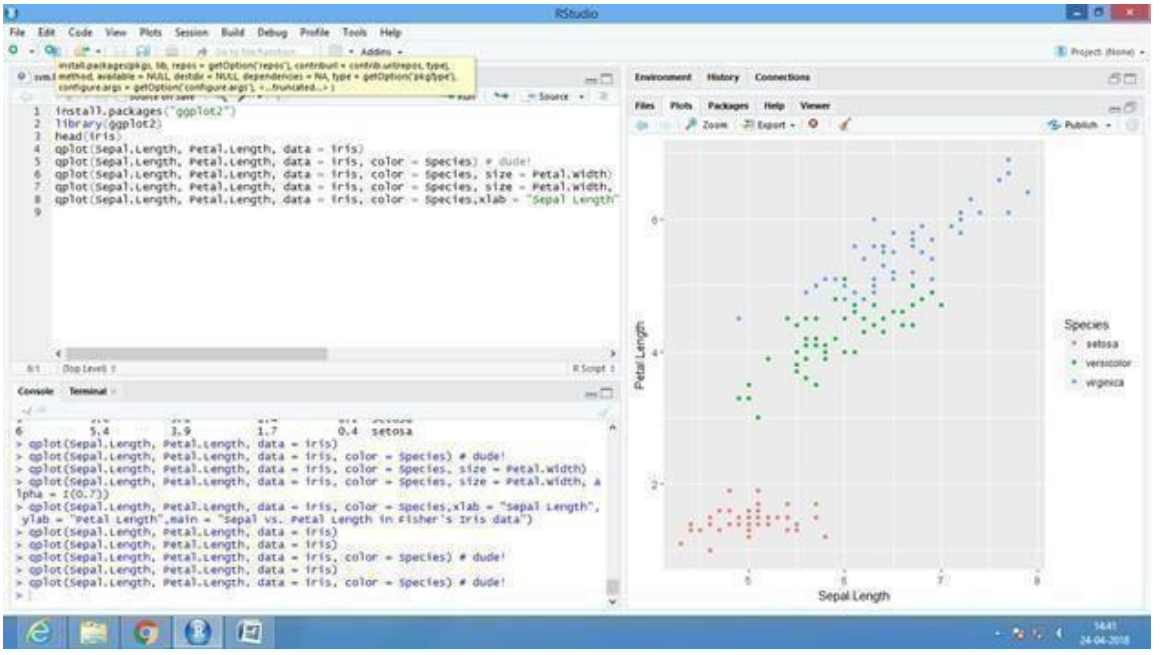
head(iris)# by default, head displays the first 6 rows

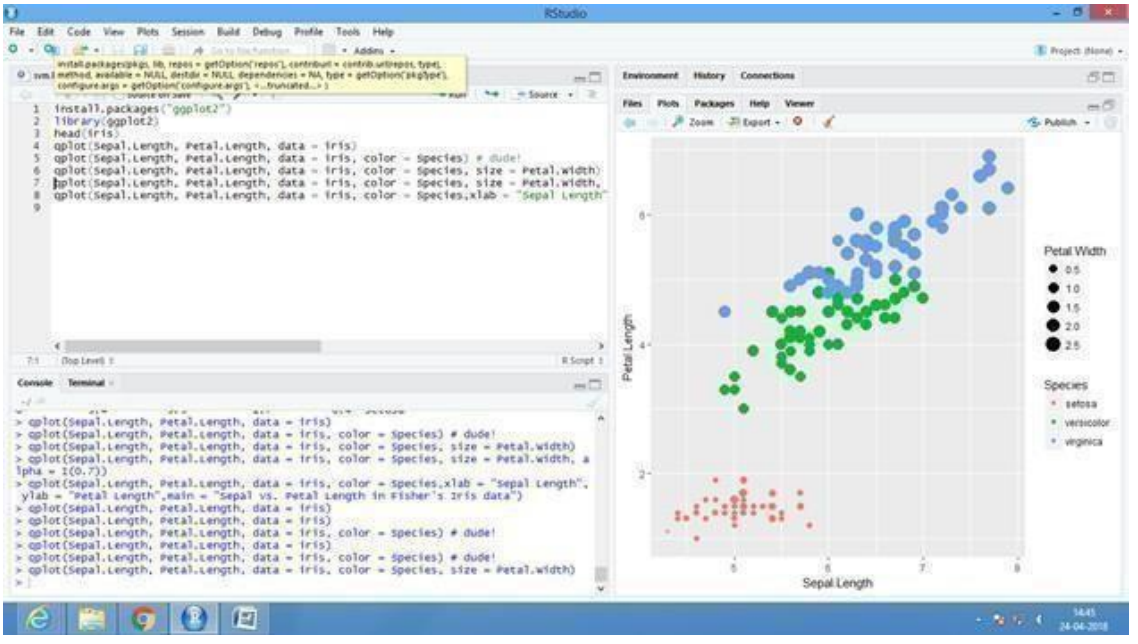
head(iris, n = 10)# we can also explicitly set the number of rows to display Let's plotSepal.Length against Petal.Length using ggplot2′s qplot()function.

**qplot(Sepal.Length,Petal.Length,data=iris)** #Plot Sepal.Length vs. Petal.Length, using data from the `iris` data frame.

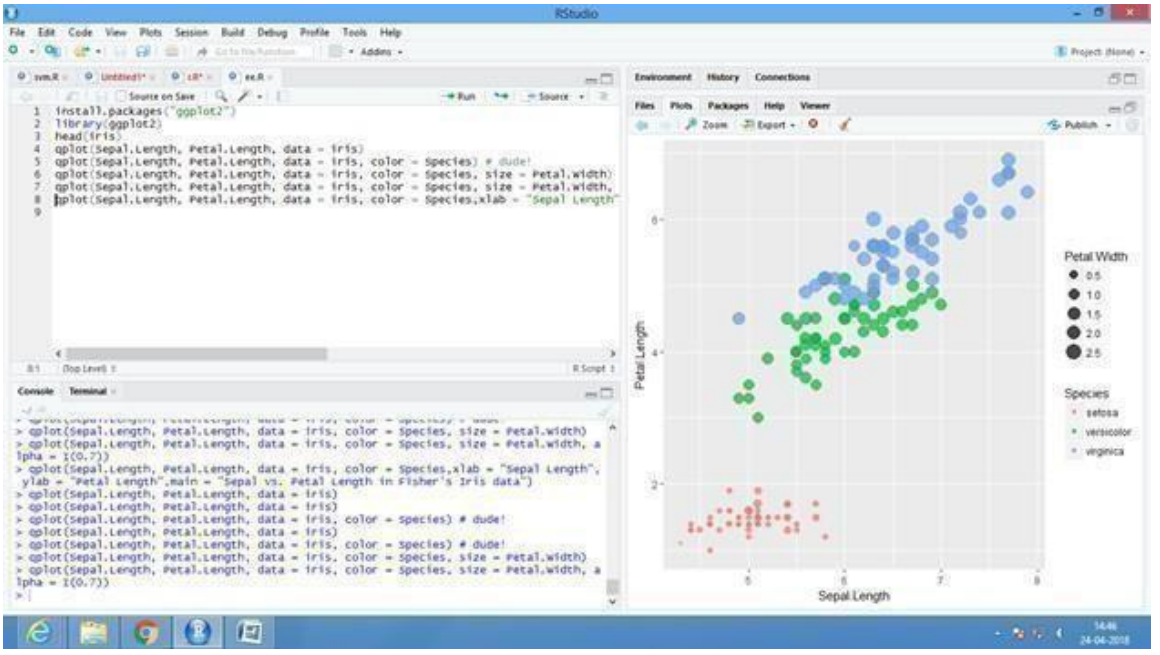


To see where each species is located in this graph, we can color each point by adding a color=Species argument.**qplot(Sepal.Length, Petal.Length, data = iris, color = Species)** # dude!

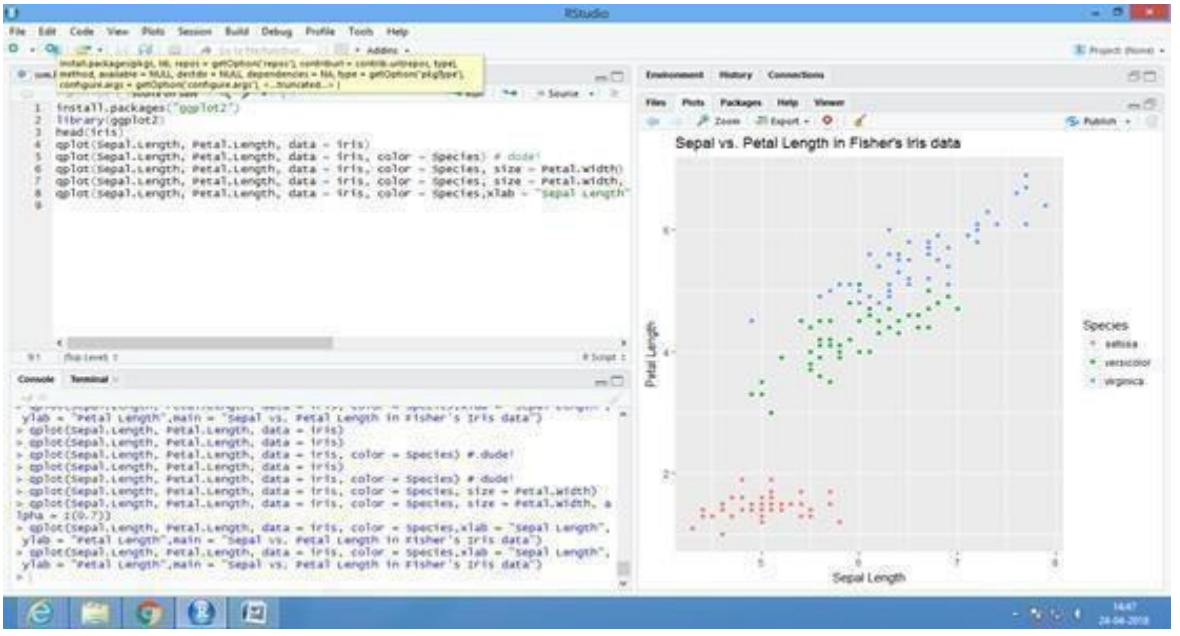



**qplot(Sepal.Length,Petal.Length,data=iris,color=Species,size=Petal.Width,alpha= I(0.7))**# By setting the alpha of each point to 0.7, we reduce the effects of overplotting.



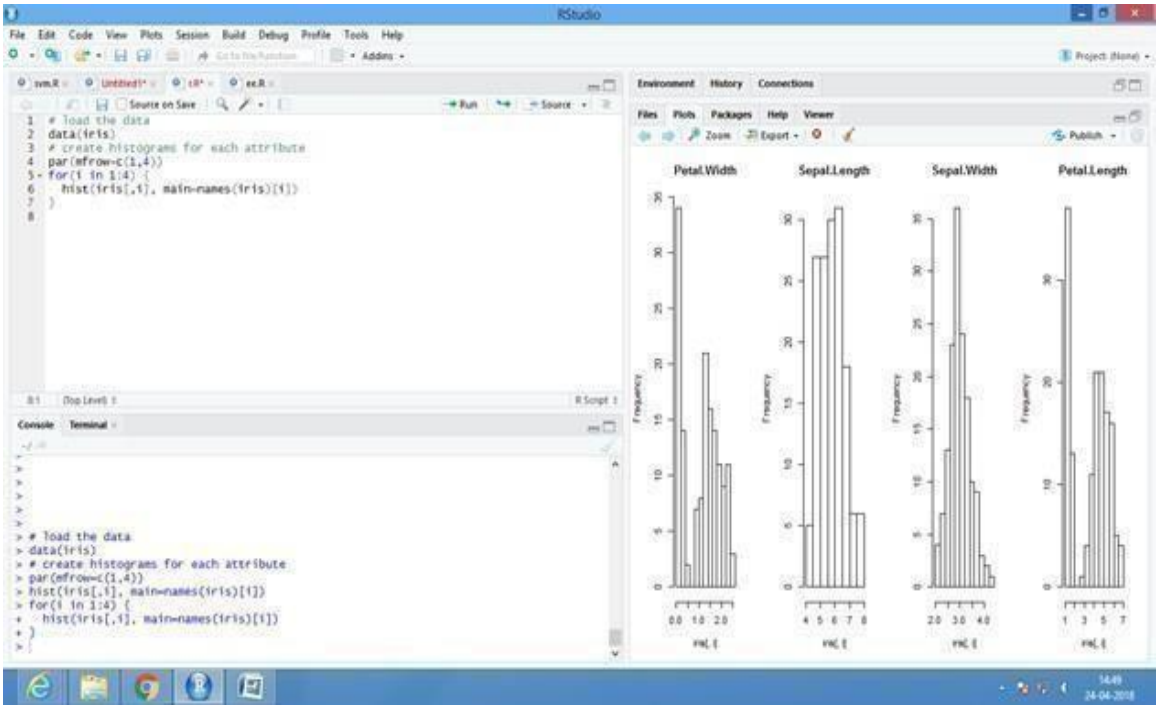

Finally, let's fix the axis labels and add a title to the plot.**qplot(Sepal.Length, Petal.Length, data=iris,color=Species,xlab="SepalLength",ylab="PetalLength",main="Sepal vs. Petal Length in Fisher's Iris data")**



## Univariate Visualization

Univariate plots are plots of individual attributes without interactions.The goal is learn something about the distribution, central tendency and spread of each attributes.

```
#loadthedata
data(iris)
#createhistogramsforeachattribute
par(mfrow=c(1,4))
for(i in 1:4) {
        hist(iris[,i], main=names(iris)[i])
}
```

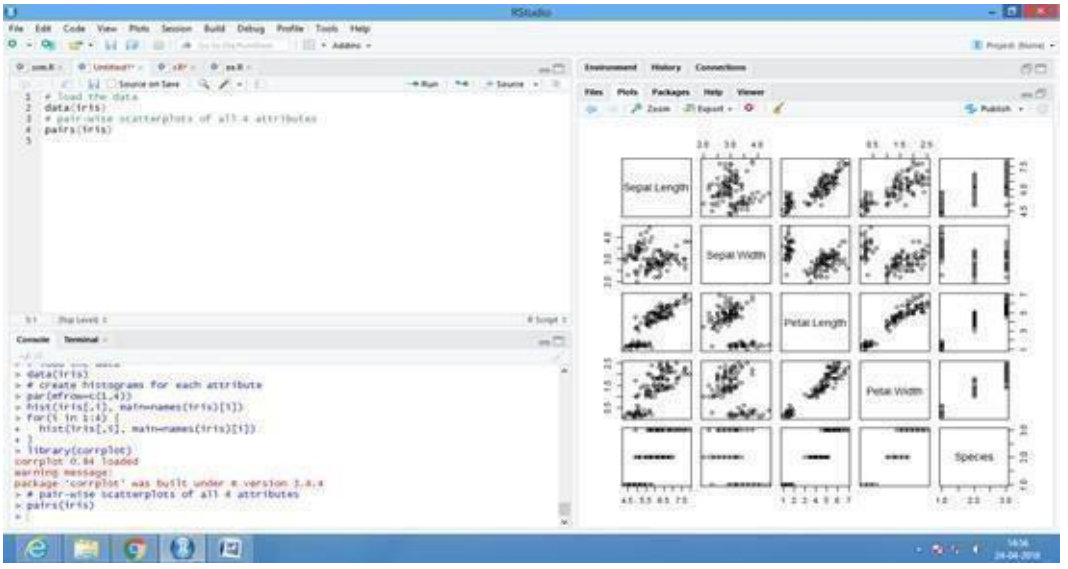



## Scatterplot Matrix

A scatterplot plots two variables together,one on each of the x and y axes with points showing the interaction.

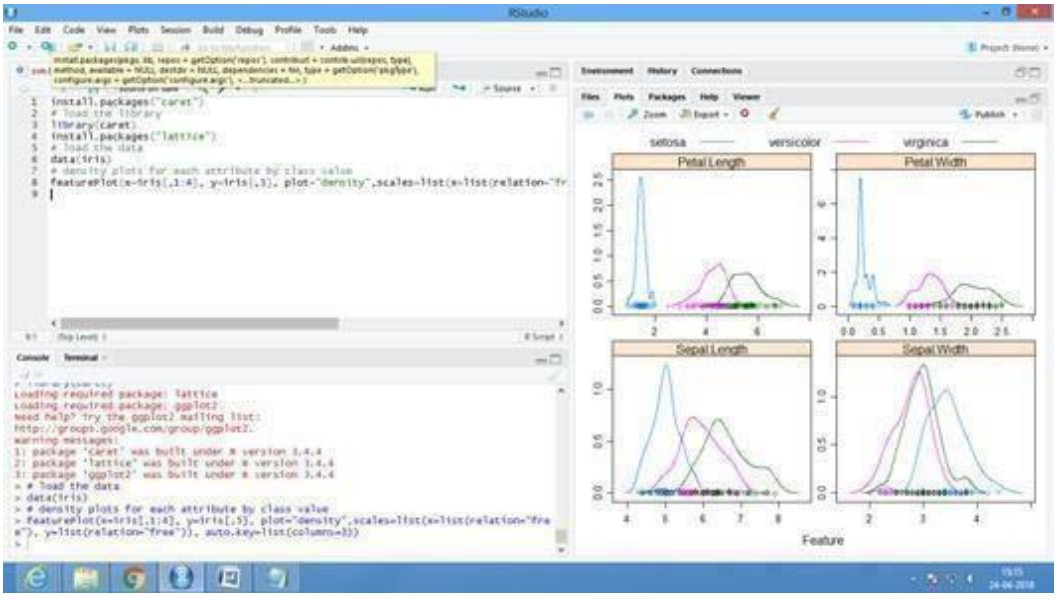
```
# load the data

data(iris)#pair-wisescatterplotsof all4attributes -pairs(iris)
```




**Density Plots:**

DensityPlots(densityplotsforshort)summarizethedistributionofthedata.#loadthe library-**library(caret),**# load the data-**data(iris)**,# density plots for each attribute by class value-featurePlot(x=iris[,1:4], y=iris[,5], plot="density", scales=list(x=list(relation="free"), y=list(relation="free")), auto.key=list(columns=3))




**RESULT:**

Thus the code for visualizing data using any plotting framework has been successfully executed.

NAME: SABITHA C
REG NO: 2303412510422307

**EX.NO: 7 IMPLEMENT AN APPLICATION THAT STORES BIG DATA IN HBASE/ MONGODB USING HADOOP/ R**

**DATE:**

**AIM:**

To implement an application that stores big data in MongoDB using Hadoop

**ALGORITHM:**

Step 1: Install MongoDB for Windows 64
bit OS. Step 2: Open Command Prompt.
Step 3: Create a database.
Step 3: Execute mongod.exe file
Step 4: After connection is established, Open another Command Prompt and Execute mongo.exe file.
Step 5: Execute the commands to create a database
table. Step 6: Execute the commands to display the
rows in table.

**PROCEDURE:**

Step 1: Install MongoDB for Windows 64 bit

https://fastdl.mongodb.org/win32/mongodb-win32-x86_64-3.2.4-si

gned.msi

Step 2: Choose Setup type as Complete

Step 3: Choose the Install button
Step 4: Proceed with the following

commands open cmd promt

C:\Users\placement

>cd .. C:\Users>cd ..

C:\>md\data\db

//A subdirectory or file \data\db

created. C:\>cd Program Files

C:\Program Files>MongoDB\Server\3.2\bin\mongod.exe

2024-07-31T14:18:32.223+0530 I CONTROL [initandlisten] MongoDB starting : pid=9568 port=27017 dbpath=C:\data\db\ 64-bit host=SYS-C18-PC

2024-07-31T14:18:32.225+0530 I CONTROL [initandlisten] targetMinOS: Windows Vista/Windows Server 2008

2024-07-31T14:18:32.226+0530 I CONTROL [initandlisten] db version v3.2.4

2024-07-31T14:18:32.226+0530 I CONTROL [initandlisten] git version: e2ee9ffcf9f5a94fad76802e28cc978718bb7a30

2024-07-31T14:18:32.227+0530 I CONTROL [initandlisten] allocator:

tcmalloc 2024-07-31T14:18:32.228+0530 I CONTROL [initandlisten]

modules: none 2024-07-31T14:18:32.229+0530 I CONTROL

[initandlisten] build environment: 2024-07-31T14:18:32.230+0530 I

CONTROL [initandlisten] distarch:

x86_64

2024-07-31T14:18:32.230+0530 I CONTROL [initandlisten]

target_arch: x86_64 2024-07-31T14:18:32.231+0530 I CONTROL

[initandlisten] options: {}

2024-07-31T14:18:32.242+0530 I STORAGE [initandlisten] wiredtiger_open config: create,cache_size=4G,session_max=20000,eviction=(threads_max=4),config_base=false,

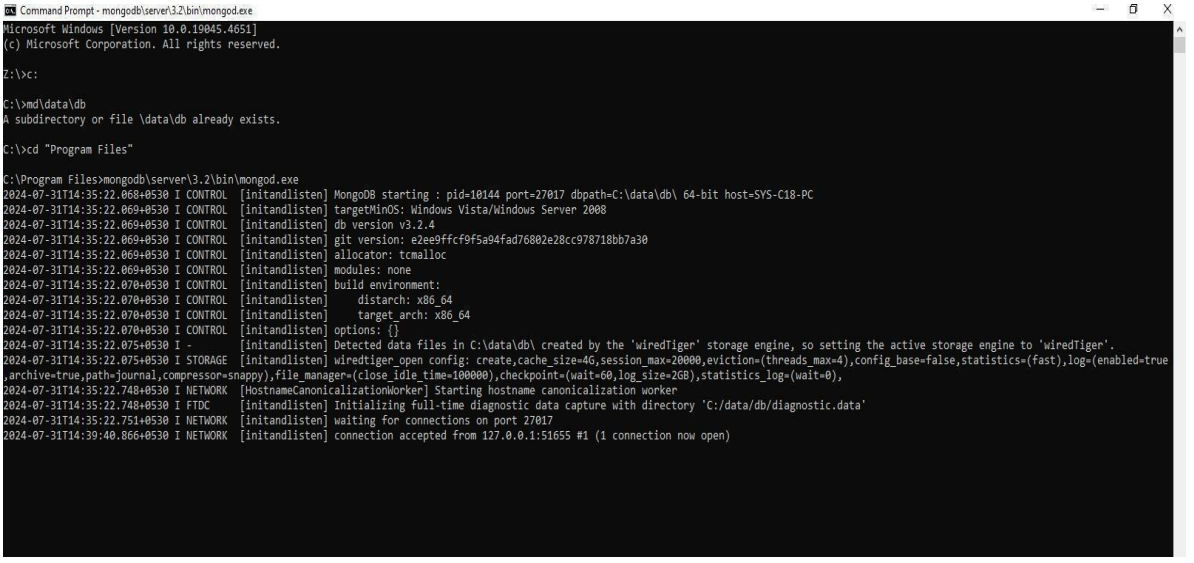
statistics=(fast),log=(enabled=true,archive=true,path=journal,compressor=snappy),file_manager
=(close_idle_time=100000),checkpoint=(wait=60,log_size=2GB),statistics_log=(wait=0),

2024-07-31T14:18:32.812+0530 I NETWORK [HostnameCanonicalizationWorker] Starting hostname canonicalization worker

2024-07-31T14:18:32.812+0530 I FTDC [initandlisten] Initializing full-time diagnostic data capture with directory 'C:/data/db/diagnostic.data'

2024-07-31T14:18:32.936+0530 I NETWORK [initandlisten] waiting for connections on port 27017

2024-07-31T14:19:39.801+0530 I NETWORK [initandlisten] connection accepted from 127.0.0.1:51473 #1 (1 connection now open)




Open another cmd prompt C:\Users\placement>cd/ C:\>cd Program Files

C:\Program Files>mongodb\server\3.2\bin\mongo.exe MongoDB shell version: 3.2.4

connecting to: test

> use mkyong switched to db mkyong

2024-07-31T14:39:20.258+0530 I NETWORK [thread1] Socket say send() errno:10053 An established connection was aborted by the software in your host machine. 127.0.0.1:27017

```
> db.users.insert({username:"prabu",password:"12345"})

2024-07-31T14:39:40.847+0530 I NETWORK [thread1] trying reconnect to
127.0.0.1:27017
(127.0.0.1) failed

2024-07-31T14:39:54.027+0530 I NETWORK [thread1] reconnect 127.0.0.1:27017
(127.0.0.1)
ok

WriteResult({ "nInserted" : 1 })

> show dps

2024-07-31T14:40:32.321+0530 E QUERY        [thread1] Error: don't know how to

show [dps] : shellHelper.show@src/mongo/shell/utils.js:803:11

shellHelper@src/mongo/shell/utils.js:594:15

@(shellhelp2):1:1

> db.users.find()

{ "_id" : ObjectId("66a9fb6b80fcd997965e9157"), "username" : "prabu", "password"
: "12345"
}

{ "_id" : ObjectId("66a9ff54f8f276937ba87e25"), "username" : "prabu", "password" :
"12345" }

>       db.users.remove({username:"prabu"}) WriteResult({ "nRemoved" : 2 })

> db.users.find()

>       db.users.insert({username:"prabu", password:"12345"}) WriteResult({

"nInserted" : 1 })

>       db.users.insert({username:"pyasodha", password:"12345"}) WriteResult({

"nInserted" : 1 })
```

> db.users.find()

{ "_id" : ObjectId("66aa0019f8f276937ba87e26"), "username" : "prabu", "password" : "12345" }

{ "_id" : ObjectId("66aa0082f8f276937ba87e27"), "username" : "pyasodha", "password" : "12345" }

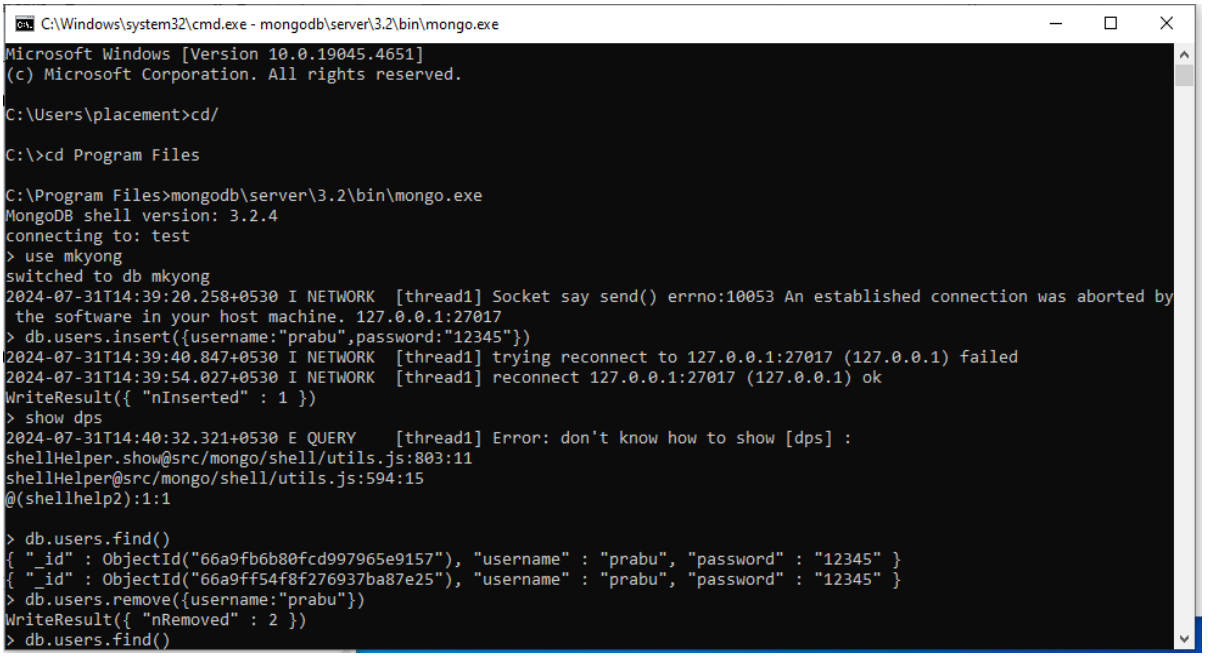
> db.users.remove({usernam

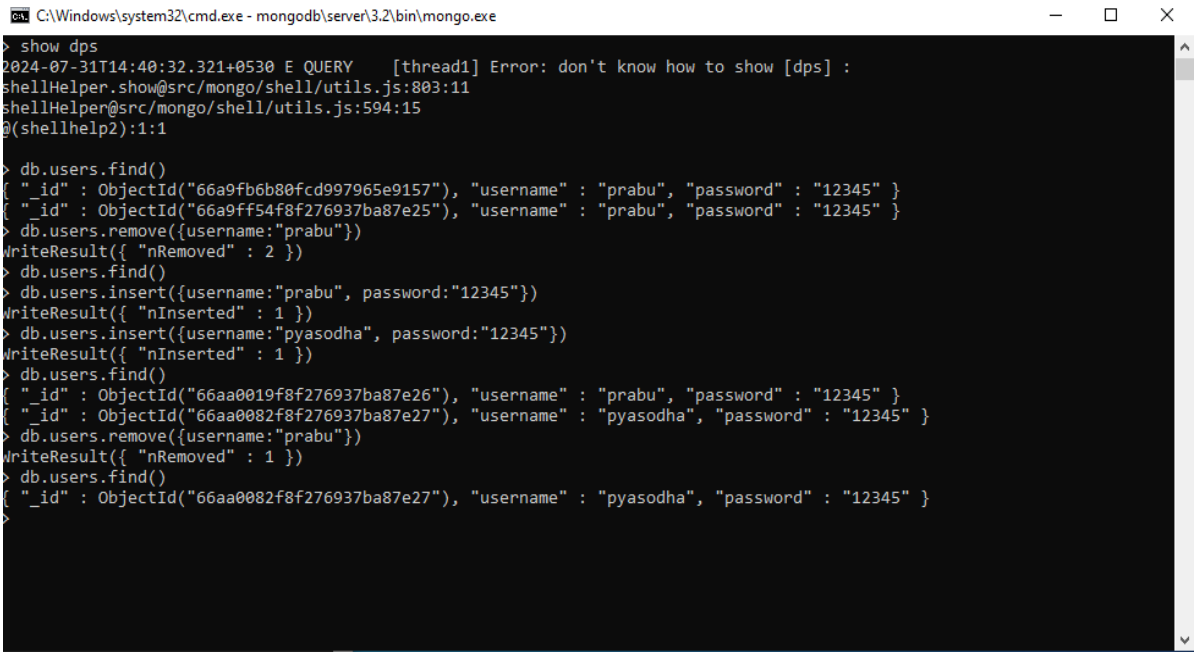
e:"prabu"}) WriteResult({

"nRemoved" : 1 })

> db.users.find()

{ "_id" : ObjectId("66aa0082f8f276937ba87e27"), "username" : "pyasodha", "password" : "12345" }






**RESULT:**

Thus the application is implemented using MongoDB in Hadoop.

## EX.NO:8 APACHE SPARK – INSTALLATION & RDD WORD COUNT

**DATE:**

**AIM**

To install and configure Apache Spark and perform **Word Count using RDDs**.

**ALGORITHM:**

**Step 1:** Extract Spark, move it to /usr/local/spark, add Spark to PATH, and start using spark-shell.

**Step 2:** Create input.txt and enter:
hello spark hello world spark apache spark
**Step 3:** In Spark Shell, load the file using:
val input = sc.textFile("file:///home/cloud/input.txt")

**Step 4:** Use flatMap() to separate the sentence into individual words.

**Step 5:** Use map() and reduceByKey() to count each word.
apache = 1, hello = 2, world = 1, spark = 3
**Step 6:** Save Output:
Use saveAsTextFile() to save the result in /home/cloud/output.
The _SUCCESS file indicates successful completion.

**PROCEDURE:**

**SPARK INSTALLATION**

1. **Open Terminal** Open the Terminal. Ctrl + Alt + T

**2. Go to Downloads**
cd ~/Downloads

**3. Extract the Spark file**
tar -xvzf spark-1.3.1-bin-hadoop2.6.tgz

**4. Move Spark to the required location**
sudo mv spark-1.3.1-bin-hadoop2.6 /usr/local/spark

**5. Check the installation**
ls /usr/local/spark

You should see folders such as:

bin conf
examples lib
sbin

**SPARK LOCATION**
Spark is installed in:/usr/local/spark/

The Spark Shell is located at:/usr/local/spark/bin/spark-shell

**6. Add Spark to PATH**
export PATH=$PATH:/usr/local/spark/bin

**7. Reload the settings**
source ~/.bashrc

**8. Start Spark**
spark-shelL

If Spark starts successfully, you will see the Spark version and a prompt like: scala>
You will also see:

sc sqlContext

**CREATE THE INPUT FILE**

**9. Exit Spark Shell**
:quit

**10. Go to the Home directory**
cd ~

**11. Create input.txt**
gedit input.txt

Type:
hello spark hello world spark apache spark Save the file and exit Nano.
The file is created in:/home/cloud/input.txt This file contains the text that Spark will count.

**12. Check the file**
ls

You can also find its location using:

find ~ -name "input.txt" 2>/dev/null

**PEFORM WORD COUNT**

**13. Start Spark Shell again**
spark-shell

**14. Read the input file**
val input = sc.textFile("file:///home/cloud/input.txt")

**USE THE FULL FILE PATH**
Use:File:///home/cloud/input.txt Do not use only:input.txt
In our experiment, using the short path caused a localhost:9000 connection error.

**15. Display the input**
input.collect()

The input text will be displayed.

**16. Split the text into words**
val words = input.flatMap(line => line.split(" ")) Check the words:
words.collect()

**17. Convert words into key-value pairs**
val pairs = words.map(word => (word, 1)) Check the pairs:
pairs.collect()

The words are now in this form:

(hello,1)
(spark,1)
(world,1)
Each word gets the value 1.

**18. Count each word**
val cleanCounts = pairs.filter(_._1 != "").reduceByKey(_ + _) Display the result:
cleanCounts.collect() Expected result:
(apache,1) (hello,2)
(world,1)
(spark,3)

**SAVE THE RESULT**

**19. Save the word count**
cleanCounts.saveAsTextFile("file:///home/cloud/output")

**OUTPUT**
Spark saves the result inside the output directory.

**20. Exit Spark Shell**
:quit

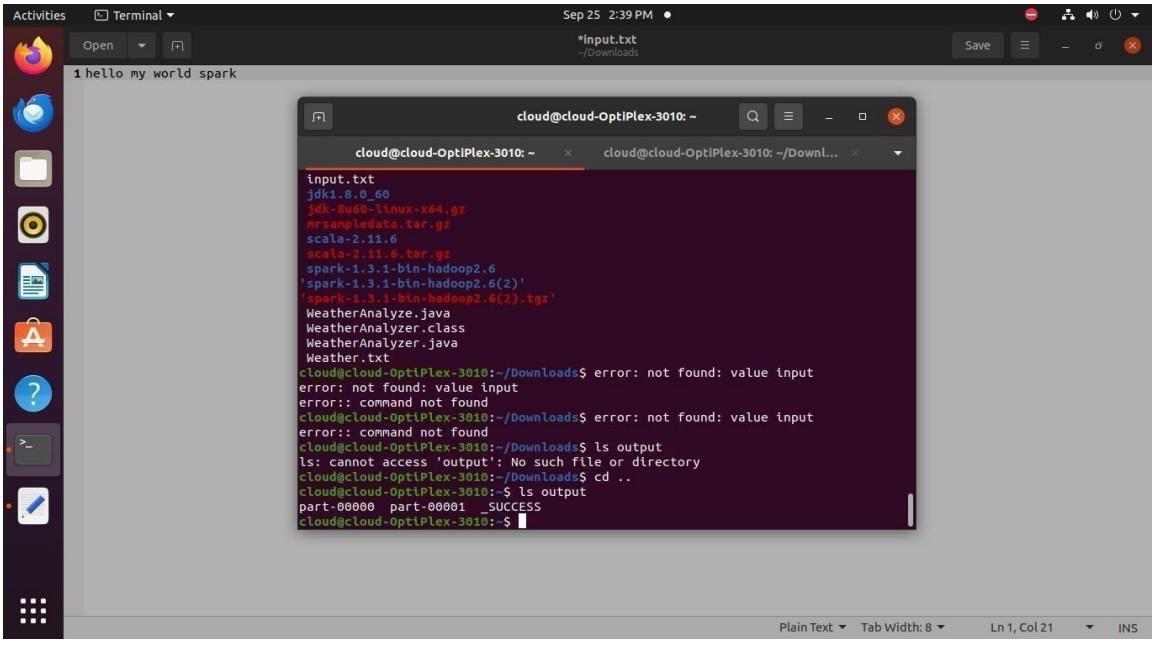
**21. Check the output**

ls output

You should see files similar to:

part-00000 part-00001
_SUCCESS

## OUTPUT




## RESULT

Apache Spark was installed and configured successfully. The input text was processed using RDD operations, the frequency of each word was calculated, and the result was saved successfully in the output directory.